

Politecnico di Torino

POLITECNICO DI TORINO

Corso di laurea in
Ingegneria Matematica

Computational linear algebra for large scale problems - Homework 3 - Methods for computing eigenvalues

Lucia Ghezzi s325951
Giuliana Paradiso s319688

15 Gennaio 2025

Contents

1 Introduction **3**
1.1 Test parameters . . . 3

2 Power method **3**
2.1 Algorithm . . . 4

3 Inverse power method **4**
3.1 Algorithm . . . 4

4 Shifting **4**
4.1 Algorithm . . . 5

5 Deflation **5**
5.1 Algorithm . . . 6

6 Test results for power method and inverse power method **6**
6.1 Computing the first and last eigenpairs . . . 6
6.2 Computing the second and second-last eigenpairs . . . 8
6.3 Computing all the eigenpairs . . . 10

7 QR method **12**
7.1 General QR method . . . 12
7.2 Reduction to upper Hessenberg form . . . 13
7.3 Algorithm for the reduction to upper Hessenberg form . . . 15
7.4 QR algorithm . . . 15
7.5 Shifted QR method . . . 15
7.6 Algorithm for shifted QR . . . 16

8 Test results for QR method **16**
8.1 Test matrix B_1 . . . 17
8.2 Test matrix B_2 . . . 17

9 Lanczos method **18**
9.1 Algorithm Lanczos method . . . 20

10 Test results for Lanczos method **20**
10.1 Test matrix C_1 . . . 21
10.2 Sparse matrix C_2 . . . 21

11 Singular value decomposition **22**
11.1 Bidiagonalization algorithm . . . 23
11.2 SVD algorithm . . . 24

12 Test results for SVD method **24**
12.1 Test matrix D_1 . . . 25

13 Conclusions **25**

List of Tables

1 Test results for the computation of the first and last eigenpairs of matrix A . 6
2 Test results for the computation of the second and second-last eigenpairs of A 8
3 Test results for the computation of all the eigenpairs of A 10
4 Test results for the computation of all the eigenpairs of H 11
5 Test results for the computation of all the eigenpairs of T 11
6 Test results for the computation of all the eigenpairs of D 11
7 Test results for the computation of the eigenvalues of B_1 17
8 Test results for the computation of the eigenvalues of B_2 17
9 Test results for the computation of the eigenvalues of C_1 21
10 Test results for the computation of the eigenvalues of C_2 21
11 Test results for the computation of the singular values of D_1 25

List of Figures

1 Sequence of λ_k converging to the first eigenvalue λ_1 of A 7
2 Sequence of λ_k converging to the last eigenvalue λ_n of A 7
3 Sequence of λ_k converging to the second eigenvalue λ_2 of A 8
4 Sequence of λ_k converging to the second eigenvalue λ_2 of A 9
5 Sequence of λ_k converging to the second-last eigenvalue λ_{n-1} of A 9
6 Matrix obtained after applying general QR algorithm to B_1 17
7 Symmetric matrix obtained after applying general QR algorithm to B_2 . . . 18
8 Tridiagonal matrix obtained after applying Lanczos algorithm to C_1 21
9 Bidiagonal matrix similar to to D_1 25

1 Introduction

This report seeks to illustrate several commonly employed methods for obtaining the eigenvalues and eigenvectors of square matrices, specifically the Power Method and Inverse Power Method (combined with shifting and deflation), the Lanczos method and the QR method. The Singular Decomposition method for the computation of singular values and singular vectors of a rectangular matrix was analyzed as well.
For each of these methods, we first present a brief description of the mathematical theory behind the method, then the pseudo-code for its implementation using the Matlab software. Finally, to evaluate the methods in terms of accuracy of the results and execution time, we report the outcomes obtained by testing the methods on some random test matrices.

1.1 Test parameters

When testing the efficiency of all the implemented methods, the following parameters were set:

- maxIter = 10000: maximum number of iterations for iterative methods
- relTol = 1e-10: stopping criteria tolerance for iterative methods
- tol_eigval = 1e-07: tolerance to check the correctness of eigenvalues/singular values
- tol_eigvec = 1e-04: tolerance to check the correctness of eigenvectors/singular vectors

2 Power method

Definition 1. *A dominant eigenvalue of the matrix A is an eigenvalue λ whose magnitude is greater than all other eigenvalues of A. If it exists, an eigenvector associated to λ is called a dominant eigenvector.*

Definition 2. *The Rayleigh quotient, for a given matrix $A \in \mathbb{R}^{(n,n)}$ and a given non-zero vector $v \in \mathbb{R}^n$ is defined as:*

$$R_A(v) = \frac{v^T A v}{||v||_2^2} \tag{1}$$

If A is symmetric and positive definite, given x_1 eigenvector of A:

$$R_A(x_i) = x_i^T A x_i = x_i^T(\lambda_i x_i) = \lambda_i(x_i^T x_i) = \lambda_i \tag{2}$$

Theorem 1.

- $\lim_{k\to\infty} \frac{v^{(k)}}{||v^{(k)}||_2} = (-1)^s x_1 \quad$ *where* $s = sign(\lambda_1)sign(\alpha_1)$
- $\lim_{k\to\infty} R_A(v^{(k)}) = \lambda_1 \quad$ *where λ_1 is the dominant eigenvalue.*

The motivation behind Power Iteration is that multiplication by a matrix tends to move vectors toward the dominant eigenvector direction. To find the dominant eigenvector of the matrix A, begin with an initial vector. Each iteration consists of normalizing the current vector and multiplying by A. The Rayleigh quotient, given an approximate eigenvector, is the best approximate eigenvalue.

2.1 Algorithm

Input: A, $v^{(0)}$, maxIter, relTol
1. $v^{(0)} = v^{(0)}/||v^{(0)}||_2$
2. $\lambda^{(0)} = \infty$
3. $k = 0$
4. while $k \leq maxIter$ and $|\lambda^{(k)} - \lambda^{(k-1)}| \geq relTol|\lambda^{(k)}|$
5. $\tilde{v}^{(k+1)} = Av^{(k)}$
6. $\lambda^{(k+1)} = v^{(k)}\tilde{v}^{(k+1)}$
7. $v^{(k+1)} = \tilde{v}^{(k+1)}/||\tilde{v}^{(k+1)}||_2$
8. $k = k + 1$
9. end while

3 Inverse power method

The inverse power method exploits the fact that, for a non singular matrix A $\in \mathbb{R}^{(n,n)}$ having eigenvalues $|\lambda_1| > |\lambda_2| > ... > |\lambda_n|$, its inverse A^{-1} has eigenvalues $|\mu_n| = |\frac{1}{\lambda_1}| < |\mu_{n-1}| = |\frac{1}{\lambda_2}| < ... < |\mu_1| = |\frac{1}{\lambda_n}|$. Then, to find the smallest eigenvalue of A one can compute the greatest eigenvalue of its inverse, applying the power method for eigenvalues to A^{-1}.

3.1 Algorithm

1. while $k \leq maxIter$ and $|\mu^{(k)} - \mu^{(k-1)}| > |\mu^{(k)}|relTol$:
2. $\tilde{v}^{(k+1)} = A^{-1}v^{(k)}$
3. $\mu^{(k+1)} = v^{(k)}\tilde{v}^{(k+1)}$
4. $v^{(k+1)} = \tilde{v}^{(k+1)}/||\tilde{v}^{(k+1)}||_2^2$
5. $k = k + 1$
6. end while
7. $\lambda_n = \frac{1}{\mu^{(k+1)}}$, $v_n = v^{(k+1)}$

It outputs: the smallest eigenvalue λ_n, its corresponding n-dimensional eigenvector v_n.

4 Shifting

When for a symmetric matrix $A \in \mathbb{R}^{(n,n)}$ the first eigenpair (λ_1, v_1) has already been computed, shifting method allows to simplify the task of computing the successive greatest

eigenvalue λ_2. In particular, A can be written as:

$$A = XDX^T = \sum_{j=1}^{n} \lambda_j x_j x_j^T = \lambda_1 x_1 x_1^T + \sum_{j=2}^{n} \lambda_j x_j x_j^T \tag{3}$$

where $D \in \mathbb{R}^{(n,n)}$ is a diagonal matrix with main diagonal the vector of eigenvalues of $A : (\lambda_1, \lambda_2, ..., \lambda_n)$, and $X \in \mathbb{R}^{(n,n)}$ is a matrix such that the j-th column of X is equal to the eigenvector x_j associated with the eigenvalue λ_j. Then, the matrix $A_1 \in \mathbb{R}^{(n-1,n-1)}$ defined as:

$$A_1 = A - \lambda_1 x_1 x_1^T = \sum_{j=2}^{n} \lambda_j x_j x_j^T \tag{4}$$

has eigeinvalues $(\lambda_2, \lambda_3, ..., \lambda_n, 0)$ and eigenvectors $(x_2, x_3, ..., x_n, x_1)$.
If eigenvalues are different, by applying the power method to A_1, one can obtain the dominant eigenvalue and eigenvector of A_1, which correspond to the second greatest eigenvalue and eigenvector of A. The shifting method exploits this property of symmetric matrices and allows constructing iteratively a sequence of matrices $A_1, ..., A_{n-1}$, to which it is possible to apply the Power Method (if the eigenvalues have different moduli), to retrieve all the eigenpairs of a given symmetric matrix A.

4.1 Algorithm

Input: symmetric matrix $A \in \mathbb{R}^{(n,n)}$, greatest eigenpair of $A : (\lambda_1, v_1)$
1. $A_1 = A - \lambda_1 v_1 v_1^T$
Output: matrix $A_1 \in \mathbb{R}^{n-1,n-1}$

5 Deflation

When for a matrix A $\in \mathbb{R}^{(n,n)}$ the first eigenpair (λ_1, v_1) has already been computed, deflation method allows to simplify the task of computing the successive greatest eigenvalue λ_2. In particular, deflation method allows to construct a matrix $P_1 \in \mathbb{R}^{(n,n)}$ such that

$$P_1 A P_1^T = \begin{bmatrix} \lambda_1 & -b_1^T \\ 0 & A_2 \end{bmatrix}$$

where $0 \in \mathbb{R}^{(n-1)}$ and $A_2 \in \mathbb{R}^{(n-1,n-1)}$ is a matrix having eigenvalues $\lambda_2, ..., \lambda_n$. The matrix P_1 is defined as

$$P_1 = I_n - 2\frac{(v_1 + e_1^{(n)})(v_1 + e_1^{(n)})^T}{||v_1 + e_1^{(n)}||_2^2}$$

where $e_1^{(n)} \in \mathbb{R}^n$ st $e_1^{(n)} = (1, 0, ..., 0)^T$.

Then, one can compute the second greatest eigenvalue of A computing the greatest eigenvalue of A_2, λ_2.
The corresponding eigenvector can also be computed: given $v_2^{(n-1)} \in \mathbb{R}^{n-1}$, eigenvector of

A_2 corresponding to the eigenvalue λ_2, to find v_2 st $Av_2 = \lambda_2 v_2$, we need to "fill" the first component. To do so, we define $\bar{v_2} = (\alpha_1, v_2^{(n-1)})^T$ and compute $\alpha_1 = -\frac{b_1^T v_2^{(n-1)}}{\lambda_1 - \lambda_2}$. Then $v_2 = P_1 \bar{v_2}$.
To obtain the second smallest eigenvalue of A, (if A has eigenvalues different in modulus) it is possible apply this procedure to A^{-1} and eigenpair (λ_n, v_n), to get a (n-1)x(n-1) matrix having eigenvalues $|\mu_2| = |\frac{1}{\lambda_{n-1}}| > |\mu_3| = |\frac{1}{\lambda_{n-2}}| > ... > |\mu_n| = |\frac{1}{\lambda_1}|$, to which *inverse power method* can therefore be applied to find λ_{n-1}, second smallest eigenvalue of A and its corresponding eigenvector v_{n-1}.

5.1 Algorithm

Input: matrix A of size n, the first eigenvector v_1
1. define $e_1^{(n)}$ and compute $v_1 + e_1^{(n)}$
2. compute P_1
3. compute $P_1 A P_1^T$
4. get $A_2 = P_1 A P_1^T(2 : end, 2 : end)$
5. get $b_1 = P_1(1, 2 : end)$
Outputs: P_1, A_2, b_1, $P_1 A P_1^T$

6 Test results for power method and inverse power method

This section presents test results for the computation of the eigenpairs of a random square matrix A, obtained exploiting Power Method and Inverse Power Method. The random matrix $A \in \mathbb{R}^{(9,9)}$, satisfies:

$$|\lambda_1| > |\lambda_2| > ... > |\lambda_{n-1}| > |\lambda_n| \tag{5}$$

6.1 Computing the first and last eigenpairs

The fist step for computing all the eigenpairs of A is the computation of the fist and last eigenpair, whose results are hold in the following table and figures.

	power method	**inverse power method**
convergence reached	yes	yes
correctness of eigenvalue*	yes	yes
execution time	7.6323e-03	6.918300e-03
num. of iterations	145	19
exp. order of convergence	9.971075e-01	1.006076e+00
residual of $Av - \lambda v$	8.293623e-11	1.621325e-12

Table 1: Test results for the computation of the first and last eigenpairs of matrix A

*with a tolerance of e-07

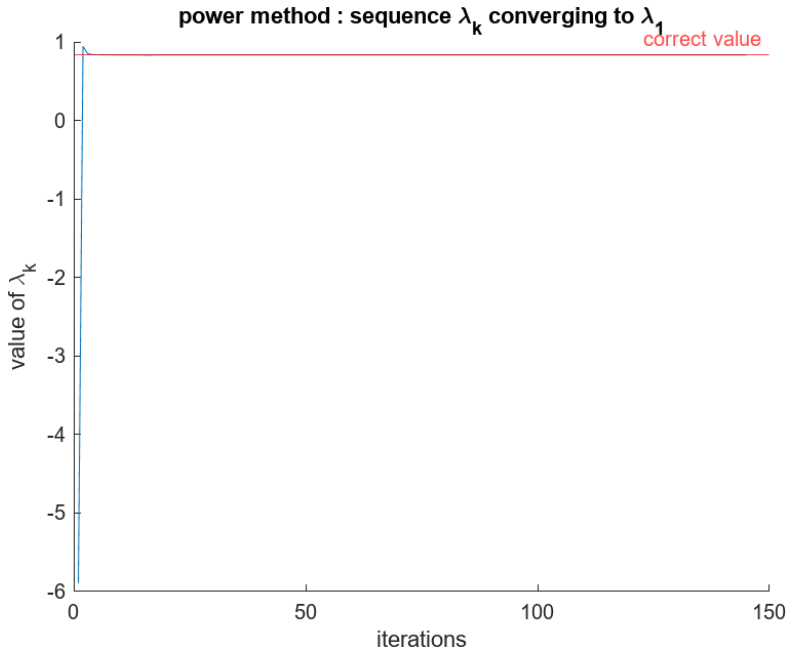


Figure 1: Sequence of λ_k converging to the first eigenvalue λ_1 of A

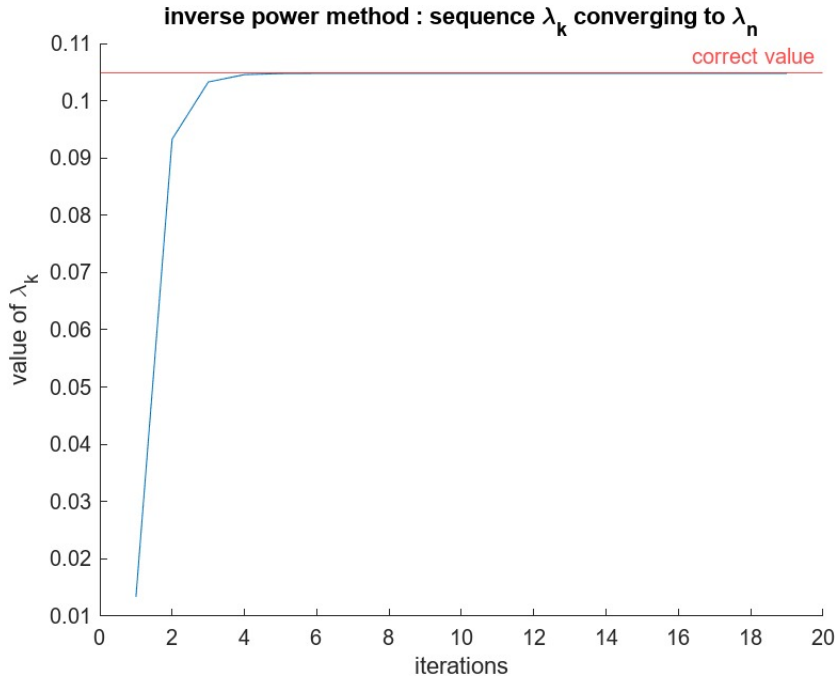


Figure 2: Sequence of λ_k converging to the last eigenvalue λ_n of A

Test results are valuable: convergence of the sequence λ_k to the correct value of $\lambda_i,\ i = 1, n$ was reached in a short execution time, although with a larger number of iterations for Power method. The experimental order of converge is linear, consistently with theoretical results, and the accuracy of the found eigenvectors is satisfying.

6.2 Computing the second and second-last eigenpairs

After computing the first and last eigenpairs of A, it was possible to retrieve its second and second-last eigenpairs. To compute the second one, shifted Power Method and deflated Power Method were tested, while to compute the second-last eigenpair deflated Inverse Power Method was tested. Results can be visualized in the following table and figures.

	shifted P.M.	**deflated P.M.**	**deflated inv. P.M.**
convergence reached	yes	yes	yes
correctness of eigenvalue*	yes	yes	yes
execution time	2.325270e-02	1.570680e-02	1.289260e-02
num. of iterations	49	109	2902
exp. order of convergence	1.000011e+00	9.999588e-01	9.994474e-01
residual of $Av - \lambda v$	6.188377e-06	1.781375e+00	1.149827e+00

Table 2: Test results for the computation of the second and second-last eigenpairs of A

*with a tolerance of e-07

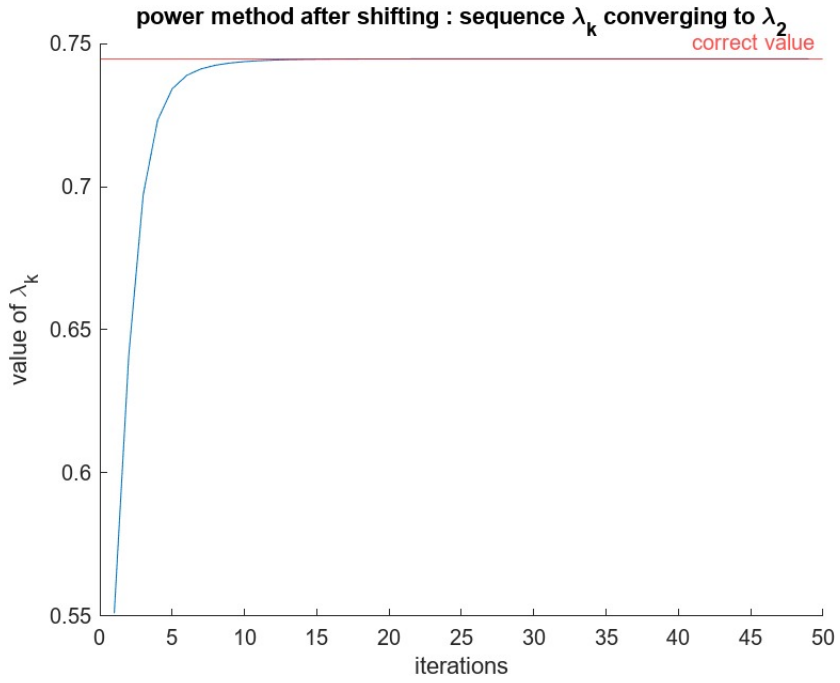


Figure 3: Sequence of λ_k converging to the second eigenvalue λ_2 of A

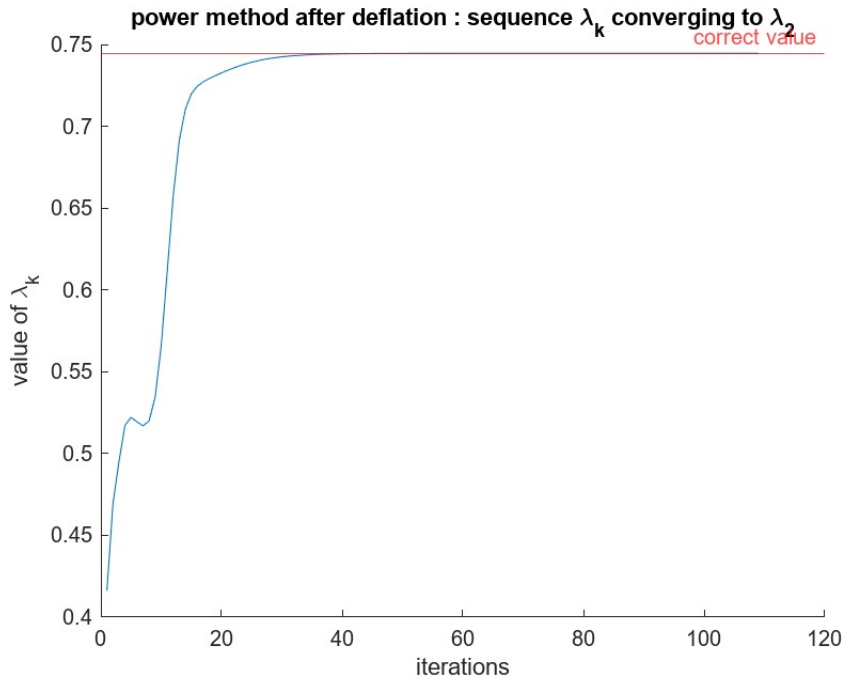


Figure 4: Sequence of λ_k converging to the second eigenvalue λ_2 of A

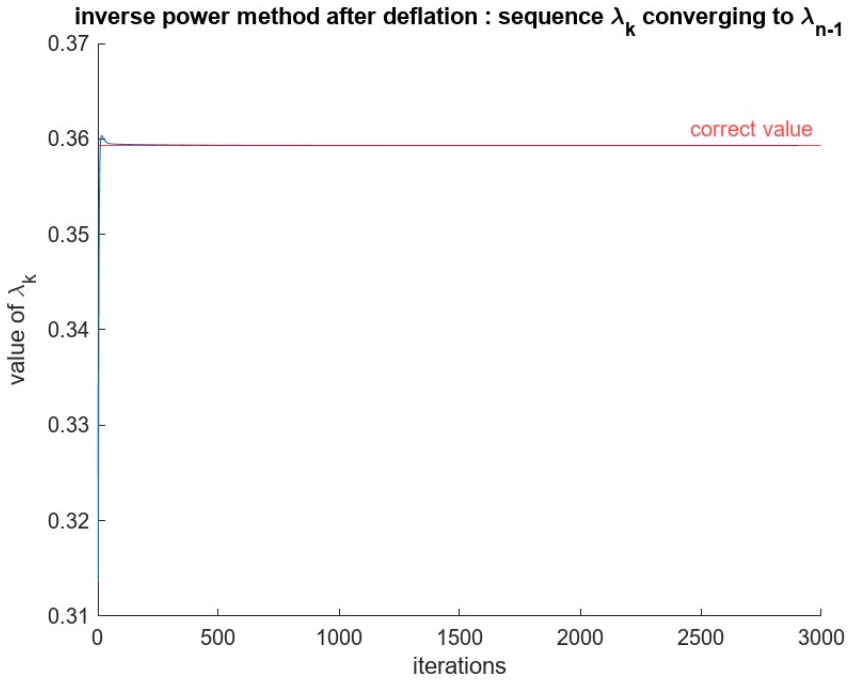


Figure 5: Sequence of λ_k converging to the second-last eigenvalue λ_{n-1} of A

The shifted Power Method and deflated Power Method obtain similar results for the computation of the second eigenvalue λ_2, providing the correct value with a tolerance of 10^{-07}, although the deflated Power Method needs a larger number of iterations. For the computation of the second eigenvector, the shifted Power Method obtains a significantly more accurate value, resulting in it being preferable to the deflated Power Method. The deflated Inverse Power Method performs well in the computation of the second-last eigenvalue λ_{n-1}, obtaining the correct result with a tolerance of 10^{-07}, while obtaining a less accurate result of the corresponding eigenvector.

6.3 Computing all the eigenpairs

Finally, tests were carried out to compute all eigenpairs of the matrix in an iterative way, from the first to the last pair, and viceversa. For this aim, the following three self-implemented functions were used and compared:

- *compute_eigenpairs*, which uses the shifted Power Method (to compute eigenpairs from the first to the last one).
- *compute_m_greatest_eigenvalues*, which uses the deflated Power Method (to compute eigenpairs from the first to the last one).
- *compute_m_smallest_eigenvalues*, which uses the deflated Inverse Power Method (to compute eigenpairs from the last to the first one).

	shifted P.M.	**deflated P.M.**	**deflated inv. P.M.**
correctness of eigenvalues*	yes	yes	yes
execution time	4.762100e-03	7.454900e-03	1.768970e-02
max residual of $Av_i - \lambda_i v_i$	1e-04	1.301403e+01	2.872173e+00

Table 3: Test results for the computation of all the eigenpairs of A

*with a tolerance of e-07

In order to evaluate the functions performances "on better" matrices, tests were also carried out using:

1. an upper Hessenberg matrix $H \in \mathbb{R}^{(9,9)}$;
2. a tridiagonal matrix $T \in \mathbb{R}^{(9,9)}$;
3. and a diagonal matrix $D \in \mathbb{R}^{(9,9)}$

All matrices were built to be similar to the previously tested A, therefore satisfy the different eigenvalues condition. Results can be visualized in the following tables.

	shifted P.M.	**deflated P.M.**	**deflated inv. P.M.**
correctness of eigenvalues*	-	yes	yes
execution time	-	2.114840e-02	2.950420e-02
max residual of $Av_i - \lambda_i v_i$	-	1.405559e+01	1.407806e+01

Table 4: Test results for the computation of all the eigenpairs of H

*with a tolerance of e-07

	shifted P.M.	**deflated P.M.**	**deflated inv. P.M.**
correctness of eigenvalues*	yes	yes	yes
execution time	5.081400e-03	9.579000e-03	1.980770e-02
max residual of $Av_i - \lambda_i v_i$	less than 1e-4	4.070957e-01	5.319271e-01

Table 5: Test results for the computation of all the eigenpairs of T

*with a tolerance of e-07

	shifted P.M.	**deflated P.M.**	**deflated inv. P.M.**
correctness of eigenvalues*	yes	yes	yes
execution time	6.015400e-03	1.375590e-02	1.639590e-02
max residual of $Av_i - \lambda_i v_i$	less than 1e-4	1.040156e-03	5.896357e-01

Table 6: Test results for the computation of all the eigenpairs of D

*with a tolerance of e-07

In general, it is possible to state that the best performing method is the shifted Power Method, whose results are very satisfying in terms of execution time and of correctness of the eigenpairs. Unfortunately, it is only possible to apply it to symmetric matrices, which constitutes its major limitation.
The deflated methods provide satisfactory results in terms of execution time and correctness of eigenvalues and yield a good approximation of eigenvectors for the tridiagonal and diagonal cases, while a rough approximation of eigenvectors is obtained for the upper Hessenberg matrix, as well as for the random one.
Finally, deflated Power Method outperforms deflated Inverse Power Method under all aspects.
The results were exploited for choosing the most suitable strategy to compute the eigenvectors of the matrices returned by the QR and Lanczos algorithms (see Sections 7 and 9, respectively).
In particular, after applying the QR method and obtaining the matrix A_k similar to the input matrix A, the following strategy is adopted. We check if the eigenvalues of A_k are different: if so, to compute eigenvectors, we adopt the shifted Power Method for symmetric matrices, and the deflated Power Method for upper Hessenberg or triangular matrices.
Similarly, after applying the Lanczos method and obtaining the tridiagonal matrix T similar to A, the following strategy is adopted: we check if the eigenvalues of T are different, if so we use the shifted Power Method, otherwise we use the Matlab function eig.

7 QR method

7.1 General QR method

Given a matrix $A \in \mathbb{R}^{(n,n)}$, we define the following sequence:

$$\begin{cases} A^{(1)} = A \\ A^{(k+1)} = R_k Q_k \end{cases}$$

where Q_k, R_k are the matrices defined by the QR decomposition of $A^{(k)}$: $A^{(k)} = Q_k R_k$, where $Q_k \in \mathbb{R}^{(n,n)}$ is an orthogonal matrix ($Q^T Q = I$) and $R_k \in \mathbb{R}^{(n,n)}$ is an upper triangular matrix. Then:

$$Q_k A^{(k+1)} = Q_k R_k Q_k \Rightarrow A^{(k+1)} = Q_k^T A^{(k)} Q_k \tag{6}$$

This meaning that the matrices $A^{(k+1)}$ and $A^{(k)}$ are similar, and by induction, all the matrices of the sequence defined as above are similar, thus, they have the same eigenvalues. To retrieve the eigenvectors of A we can do:
We assume to know an eigenpair $(\lambda_i, x_i^{(k)})$. Then:

$$\begin{gathered} A^{(k)} x_i^{(k)} = \lambda_i x_i^{(k)} \\ \Downarrow \\ Q_{k-1}^T A^{(k-1)} Q_{k-1} x_i^{(k)} = \lambda_i x_i^{(k)} \\ \Downarrow \\ A^{(k-1)} Q_{k-1} x_i^{(k)} = \lambda_i Q_{k-1} x_i^{(k)} \\ \Downarrow \\ x_i^{(k-1)} = Q_{k-1} x_i^{(k)} \end{gathered}$$

Applying recursively this definition we can retrieve each eigenvalue $x_i^{(k-j)} = Q_{k-j} \cdots Q_{k-2} Q_{k-1} x_i^{(k)}$.
The general QR method has a computational cost of $O(n^3)$, due to the need to compute the QR factorization at each iteration.

Theorem 2. *Let $A \in \mathbb{R}^{(n,n)}$ be a quadratic matrix such that its eigenvalues are mutually different in magnitude, thus we can number them such that: $|\lambda_1| > |\lambda_2| > ... > |\lambda_n|$. Let $X = [x_1, ..., x_n]$ be the matrix of the eigenvectors of A, where $x_1, ..., x_n$ are the said eigenvectors.*

- *If X^{-1} admits a LU factorization ($X^{-1} = LU$), then the sequence of matrices $A^{(k)}$ converges to an upper triangular matrix T, with principal diagonal the vector of eigenvalues of A, ordered in decreasing order.*
- *If A is symmetric and positive definite, then $A^{(k)}$ converges to the diagonal matrix of ordered eigenvalues.*
- *In the above setting, with the difference that the matrix X^{-1} admits a permuted factorization ($PX^{-1} = LU$), then the diagonal elements of T are a permutation of $(\lambda_1, ..., \lambda_n)$.*

Definition 3. *A matrix H is an upper Hessenberg matrix if*

$$a_{ij} = 0 \quad \forall i > j + 1 \tag{7}$$

Theorem 3. *Let $A \in \mathbb{R}^{(n,n)}$ be a quadratic matrix such that: $|\lambda_1| > |\lambda_2| > ... > |\lambda_r| = |\lambda_{r+1}| > ... > |\lambda_n|$. Then the sequence of matrices $A^{(k)}$ converges to an upper Hessenberg matrix T.*

The previous theorem states that in the most general case, the sequence of matrices $A^{(k)}$ converges to an upper Hessenberg matrix and repeated (in modulus) eigenvalues can be found by computing eigenvalues of the 2x2 blocks corresponding to the non zero entries of the subdiagonal of A_k.
The computational cost of applying the QR algorithm if all the matrices $A^{(k)}$ are upper Hessenberg is $O(n^2)$ instead of $O(n^3)$, since the QR algorithm preserves the upper Hessenberg form: in fact if $A^{(k)}$ is upper Hessenberg, also Q_k is upper Hessenberg, and so is $A^{(k+1)} = R_k Q_k$.
For this reason, a good strategy to reduce the computational cost of the algorithm is to start it with an upper Hessenberg matrix similar to A, and apply the algorithm to it.

7.2 Reduction to upper Hessenberg form

Let

$$A = \begin{bmatrix} a_{11} & \mathbf{b}_1^T \\ \mathbf{a}_1 & \mathbf{B} \end{bmatrix}$$

where a_{11}, $\mathbf{b}_1^T \in \mathbb{R}^{n-1}$, $\mathbf{a}_1 \in \mathbb{R}^{n-1}$ and $\mathbf{B} \in \mathbb{R}^{(n-1,n-1)}$.

Definition 4. *A matrix of the form*

$$P = I - 2uu^T, \quad ||u|| = 1 \tag{8}$$

is called an Householder reflector.

Let $H^{(1)} = I_{n-1} - 2\frac{a_1 a_1^T}{||a_1||_2^2} \in \mathbb{R}^{(n-1,n-1)}$ be an Householder reflector and it is used to transform the vector a_1 on a multiple of vector $e_1^{(n)}$:

$$H^{(1)} a_1 = \alpha_1 e_1^{(n)} \tag{9}$$

Householder reflectors can be used to reduce an arbitrary square matrix to Hessenberg form, with the following procedure.

First, the matrix

$$T^{(1)} = \begin{bmatrix} 1 & \mathbf{0}^T \\ \mathbf{0} & \mathbf{H}^{(1)} \end{bmatrix}$$

is defined. It is orthogonal and symmetric, therefore a change of basis matrix.

Then the following product is carried out:

$$A^{(1)} = T^{(1)}AT^{(1)} = \begin{bmatrix} a_{11} & b_1^T H(1) \\ \alpha_1 & \\ 0 & \mathbf{H}^{(1)}\mathbf{B} \\ \vdots & \\ 0 & \end{bmatrix} T^{(1)} = \begin{bmatrix} a_{11} & \mathbf{b}_2^T \\ \alpha_1 & \\ 0 & \mathbf{B}^{(1)} \\ \vdots & \\ 0 & \end{bmatrix}$$

The multiplication of T_1 from the left inserts the desired zeros in the first column of A. The multiplication from the right is necessary in order to have similarity.
At iteration k the first k columns are upper hessenberg:

$$A^{(k)} = \begin{bmatrix} a_{11}^{(k)} & \dots & \dots & \dots & \dots & \dots \\ a_{21}^{(k)} & a_{22}^{(k)} & \dots & \dots & \dots & \\ 0 & a_{32}^{(k)} & \ddots & & \dots & \dots \\ \vdots & 0 & \ddots & \ddots & & \\ \vdots & \vdots & & a_{k+1,k}^{(k)} & a_{k+1,k+1}^{(k)} & \mathbf{b}_{k+1}^T \\ 0 & 0 & 0 & 0 & \mathbf{a}_{k+1} & \mathbf{B}^{(k-1)} \end{bmatrix}$$

where $\mathbf{a}_{k+1} \in \mathbb{R}^{n-(k+1)}$, $\mathbf{B}^{k-1} \in \mathbb{R}^{(n-(k+1),n-(k+1))}$ is a block matrix that still needs to be transformed to obtain $A^{(k+1)}$. To do so, we define the Householder reflector:

$$H^{(k+1)} = I_{n-(k+1)} - 2\frac{a_{k+1}a_{k+1}^T}{||a_{k+1}||_2^2} \tag{10}$$

and the matrix

$$T^{(k+1)} = \begin{bmatrix} I_{k+1} & \mathbf{0} \\ \mathbf{0} & H^{(k+1)} \end{bmatrix}$$

and then compute the matrix product:

$$A^{(k+1)} = T^{(k+1)}A^{(k)}T^{(k+1)} = \begin{bmatrix} a_{11}^{(k+1)} & \dots & \dots & \dots & \dots & \dots \\ a_{21}^{(k+1)} & a_{22}^{(k+1)} & \dots & \dots & \dots & \\ 0 & a_{32}^{(k+1)} & \ddots & & \dots & \dots \\ \vdots & 0 & \ddots & \ddots & a_{k+1,k+1}^{(k+1)} & \\ \vdots & \vdots & & & a_{k+2,k+1}^{(k+1)} & \mathbf{b}_{k+1}^T \\ 0 & 0 & 0 & 0 & 0 & \mathbf{B}^{(k+1)} \end{bmatrix}$$

The computational cost of applying the upper Hessenberg transformation is $O(n^3)$. But after transforming the original matrix A into a similar matrix in upper Hessenberg form, each iteration of the QR method has a cost of $O(n^2)$ instead of $O(n^3)$.

7.3 Algorithm for the reduction to upper Hessenberg form

Input: $A \in \mathbb{R}^{(n,n)}$
1. for $k = 1 : n$
2. $T = I_n$
3. $a = A(k+1 : n, k)$
4. e=zeros(size(a, 1), 1)
5. $e(1) = ||a||_2$
6. $a = a + e$
7. $H = I_{n-k,n-k} + 2aa^T/||a||_2^2$
8. $T(k+1 : n, k+1 : n) = H$
9. $A_1 = TAT^T$
10. end for
Output: upper Hessemberg matrix A_1 similar to A

7.4 QR algorithm

Input: A, relTol, maxIter
1. $A^{(1)}$=similarHessemberg(A)
2. initialize the first subdiagonal $S_{k-1} = \mathbf{0} \in \mathbb{R}^{n-1}$
3. compute S_k, the first subdiagonal of $A^{(1)}$
4. initialize $prodQ_k = I_n$
5. k=1
6. while $k < maxIter$ and $||S_k - S_{k-1}||$ < relTol
7. compute Q_k, R_k QR decomposition of $A^{(k)}$
8. $A^{(k+1)} = R_k Q_k$
9. $S_{k-1} = S_k$
10. $S_k = diag(A_k, -1)$
11. $prodQ_k = prodQ_k Q_k$
12. $k = k + 1$
13. end while
Output: A_k
Then, the eigenvalues of A_k, similar to A but in a simpler form, can be computed.

7.5 Shifted QR method

The convergence of the Hessenberg QR algorithm can be improved by introducing shifts into the algorithm. This is due to the fact that the subdiagonal element $a_{i+1,1}^{(k)}$ goes to zero as $\frac{|\lambda_i+1|}{|\lambda_i|}$ and shifting makes these ratios smaller and smaller. For $i = 1, ..., n$ the following procedure can be applied:

1. run the QR method for p iterations, then define $\mu = a_{n,n}^{(p)}$
2. modify the QR sequence introducing the shift:
   compute the QR factorization of $A^{(k)}$ shifted: $VA^{(k)} - \mu I_{n,n} = Q_k R_k$
   update the sequence as $A^{(k+1)} = R_k Q_k + \mu I_{n,n}$

3. update $\mu = a_{n,n}^{(k+1)}$

By doing so, since $\mu - \lambda_n \approx 0$, $a_{n,n-1}$ goes to zero faster and when it is almost zero,

$$A^{(k)} = \begin{bmatrix} \mathbf{B} & \dots \\ 0 & a_{n,n}^{(k)} \end{bmatrix}$$

and you get the approximation $\lambda_n \approx a_{n,n}^{(k)}$ and the algorithm can be rerun on the smaller matrix $B \in \mathbb{R}^{(n-1,n-1)}$.

7.6 Algorithm for shifted QR

Input: matrix L, maxIter, relTol
1. compute A: upper Hessenberg matrix similar to L
2. initialize vector eigenvalues= $\mathbf{0} \in \mathbb{R}^n$
3. $k = 1$
4. initialize sequence $A_k = A$
5. for $i = n, ..., 2$:
6. compute new matrix $A_k = A_k(1 : i, 1 : i)$
7. save last subdiagonal element $\mu_{k,sub} = A_k(i, i-1)$
8. save last diagonal element $\mu_k = A_k(i, i)$
9. while k≤maxIter and $|\mu_{k,sub}|$ >relTol
10. $\mu_{k,sub} = A_k(i, i-1)$
11. $\mu_k = A_k(i, i)$
12. compute QR decomposition of $A_k - \mu_k I_k$
13. update $A_k : A_k = R_k Q_k + \mu_k I_k$
14. k=k+1
15. end while
16. eigenvalues(i)=μ_k
17. end for
18. eigenvalues(1)=$A_k(1, 1)$

Output: eigenvalues of A

8 Test results for QR method

This section presents test results for the QR method. Tests were carried out on the following random matrices:

- $B_1 \in \mathbb{R}^{(9,9)}$
- $B_2 \in \mathbb{R}^{(9,9)}$ symmetric and positive definite

8.1 Test matrix B_1

	QR	**QR with upper Hess. reduc.**	**shifted QR with upper Hess. reduc.**
execution time	2.523032e-01	1.530871e-01	6.577020e-02
num. of iterations	9999	9999	10000 $\forall$ shift
correctness of eigenvalues*	yes	yes	yes
max residual $Av_i - \lambda_i v_i$*	1.055657e+00	4.701780e+00	-

Table 7: Test results for the computation of the eigenvalues of B_1

*with a tolerance of e-07

4.4388	0.0541	0.2256	-0.4077	0.1586	0.6690	-0.2947	0.3261	-0.0058
0	-0.0183	-0.5547	0.1172	0.0417	-0.4821	0.0105	-0.4888	-0.1791
0	0.9347	-0.2323	-0.1962	0.1436	0.2955	-0.0537	-0.1840	0.5428
0	0.0000	-0.0000	-0.5053	0.5560	-0.0772	0.0291	0.0798	-0.2506
0	-0.0000	0.0000	-0.2474	-0.5723	-0.3565	0.0502	-0.1086	-0.1913
0	0	0	-0.0000	-0.0000	0.5983	-0.3171	-0.1917	-0.1447
0	0	0.0000	0	-0.0000	0.3915	0.1184	0.0208	-0.0782
0	0	0.0000	0	0	0	0	-0.3279	0.4441
0	0	0.0000	0	0	0.0000	-0.0000	0	0.2839

Figure 6: Matrix obtained after applying general QR algorithm to B_1

8.2 Test matrix B_2

	QR	**QR with upper Hess. reduc.**	**shifted QR with upper Hess. reduc.**
execution time	1.065841e-01,	9.743550e-02	1.511770e-02
num. of iterations	9999	9999	less than 21 per shift
correctness of eigenvalues*	yes	yes	yes
max residual $Av_i - \lambda_i v_i$*	less than 1e-4	2.022168e+01	-

Table 8: Test results for the computation of the eigenvalues of B_2

*with a tolerance of e-07

29.6347	-0.0000	-0.0000	0.0000	-0.0000	-0.0000	0.0000	0.0000	-0.0000
0	10.6132	-0.0000	-0.0000	-0.0000	0.0000	0.0000	0.0000	-0.0000
0	-0.0000	10.0395	0.0000	-0.0000	-0.0000	0.0000	-0.0000	-0.0000
0	0.0000	0.0000	9.7181	-0.0000	0.0000	0.0000	-0.0000	-0.0000
0	0.0000	-0.0000	0.0000	9.4858	-0.0000	0.0000	-0.0000	-0.0000
0	-0.0000	-0.0000	0.0000	0.0000	9.3779	-0.0000	-0.0000	-0.0000
0	0.0000	0.0000	0.0000	-0.0000	-0.0000	9.1688	0.0000	-0.0000
0	0.0000	0.0000	-0.0000	0.0000	-0.0000	-0.0000	9.0589	-0.0000
0	0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	-0.0000	9.0071

Figure 7: Symmetric matrix obtained after applying general QR algorithm to B_2

In general, the QR method yields results that align with the theoretical expectations concerning the obtained reduced matrix: it is upper hessenberg (or upper triangular) for the random matrix B_1 and diagonal for the symmetric positive definite matrix B_2.
In all cases, the computed eigenvalues are correct (up to a tolerance of the order of 10^7). It is interesting to notice that the execution time is significantly reduced with both the improvements (construction of a similar upper hessenberg and shifting). This is due to the fact that the QR decomposition of the initial matrix B_1 is more computationally expensive than the one of an upper hessenberg matrix; moreover, shifting also reduces the size of the considered matrix, which constitutes another improvement in the computational time.
The approximation of the eigenvectors is indeed less accurate with respect to the one for eigenvalues; still, it improves when the method is applied to the symmetric positive definite matrix.

9 Lanczos method

Definition 5. *Let $A \in \mathbb{R}^{(n,n)}$ be a matrix and $v \in \mathbb{R}^n$ be a non-vanishing vector. Krylov subspaces are subspaces of the form*

$$\mathcal{S}_k(A, v) = span\{v, Av, A^2v, ..., A^{k-1}v\} \tag{11}$$

From now on, they will be denoted as $\mathcal{S}_k$.

Theorem 4. *Let $A \in \mathbb{R}^{(n,n)}$ be a symmetric and positive definite matrix. Let $U_m \in \mathbb{R}^{(n,m)}$, $m \leq n$ be a rectangular matrix such that $U_m^T U_m = I_m$. Then, if λ_1 and λ_n are the maximum and minimum eigenvalues of A and μ_1 and μ_m are the the maximum and minimum eigenvalues of $U_m^T A U_m$,*

$$\mu_1 \leq \lambda_1 \quad and \quad \lambda_n \leq \mu_m. \tag{12}$$

Moreover, if $m = n$, then the matrix $U_n^T A U_n$ is similar to A and thus $\mu_1 = \lambda_1$ and $\mu_n = \lambda_n$.

Theorem 5. *Let $A \in \mathbb{R}^{(n,n)}$ be a symmetric and positive definite matrix with eigenvalues $\lambda_1 \geq \lambda_1 \geq ... \geq \lambda_n > 0$. Let $R_A(v)$ be the Rayleigh quotient of the vector $v \in \mathbb{R}^n$. Then*

$$\lambda_k = \max_{\substack{V_k \in \mathbb{R}^n \\ dim V_k = k}} \min_{\substack{v \in V_k \\ v \neq 0}} R_A(v) \quad \forall \quad k = 1, ..., n \tag{13}$$

$$\lambda_{n-k+1} = \min_{\substack{V_k \in \mathbb{R}^n \\ dim V_k = k}} \max_{\substack{v \in V_k \\ v \neq 0}} R_A(v) \quad \forall \quad k = 1, ..., n \tag{14}$$

For $k = n$ in equation 13

$$\lambda_n = \min_{v \in \mathbb{R}^n} R_A(v) \tag{15}$$

For $k = n$ in equation 14

$$\lambda_1 = \max_{v \in \mathbb{R}^n} R_A(v) \tag{16}$$

The idea behind the Lanczos method is the following. At each iteration k a Krylov subspace $\mathcal{S}_k = \text{span}\{q_1, Aq_1, A^2q_1, ..., A^{k-1}q_1\}$ is defined, and the matrix $Q_k = [q_1, q_2, ..., q_k] \in \mathbb{R}^{(n,k)}$ is computed, where the vectors $\{q_1, ..., q_k\}$ form an orthonormal basis of $\mathcal{S}_k$. Then, A and B_k are similar through Q_k: $Q_k^T A Q_k = B_k$, so when $k = n$ A and B_k have the same eigenvalues and eigenvectors of A can be retrieved. In general, for $k < n$, if $\mu_1^{(k)}$ and $\mu_k(k)$ are, respectively, the maximum and minimum eigenvalues of B_k, the following holds:

$$\mu_1^{(k)} \nearrow \lambda_1, \quad \mu_1^{(n)} = \lambda_1$$

$$\mu_k^{(k)} \nearrow \lambda_n, \quad \mu_n^{(n)} = \lambda_n$$

To find the maximum and minimum eigenvalues of B_k, it's possible to exploit the Rayleigh quotient by approximating its stationary points. Gradient descent and gradient ascent can be used for this aim. The gradient of the Rayleigh quotient is:

$$\nabla R_A = g(x) = \frac{2}{||x||_2^2}(Ax - R_A(x)x) \tag{17}$$

At iteration k, we have the subspace $\mathcal{S}_{k-1} = \text{span}\{q_1, Aq_1, A^2q_1, ..., A^{k-2}q_1\}$ and the vectors $\{q_1, ..., q_{k-1}\}$ are an orthonormal basis for it. The Lanczos method defines a strategy to compute the vectors q_i with the Lanczos tridiagonalization:
At step k, assume to have $q_1, ..., q_{k-1}$ such that:

$$Q_{k-1}^T A Q_{k-1} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & ... & 0 \\ \beta_1 & \ddots & \ddots & & \\ 0 & \ddots & \ddots & \ddots & \\ \vdots & & \ddots & \ddots & \beta_{k-1} \\ 0 & & & \beta_{k-1} & \alpha_k \end{bmatrix}$$

Try to compute q_k orthogonal to all other q_i and such that:

$$\begin{cases} q_k^T q_i = 0 \quad \forall i = 1, .., k-1 \\ Aq_{k-1} = \beta_{k-2}q_{k-2} + \alpha_{k-1}q_{k-1} + \beta_{k-1}q_k \end{cases}$$

so $q_k^T A q_k = \alpha_{k-1}$ and $q_k = \frac{1}{\beta_{k-1}}(Aq_{k-1} - \alpha_{k-1}q_{k-1} - \beta_{k-1}q_{k-2})$, where we impose
$\beta_{k-1} = ||(A - \alpha_{k-1}I)q_{k-1} - \beta_{k-1}q_{k-2}||_2$ for orthonormality.
Now check if $\beta_{k-1} \neq 0$ and, if so, define q_k and try to compute q_{k+1}, otherwise $dim(\mathcal{S}_k) = k-1$, meaning that $\mathcal{S}_k$ is invariant and the algorithm can be stopped.

When carried out in finite arithmetics, the Lanczos algorithm is affected by rounding errors.

$$\beta_{k+1} = ||Aq_{k+1}|| - ||\alpha_{k+1}q_{k+1}|| - ||\beta_k q_k|| - ||\epsilon_k|| \tag{18}$$

where ϵ_k accounts for the rounding errors up to the k-th step.
This can cause the Lanczos vectors $\{q_k\}$ not being orthogonal. To improve the accuracy of the Lanczos method is possible to apply a reorthogonalization of the vector q_k at each step k. This aim can be achieved by rescaling r_k in the following way:

$$r_k = r_k - Q_k Q_k^T r_k \tag{19}$$

9.1 Algorithm Lanczos method

Input: $A \in \mathbb{R}^{n,n}$, $q_1 \in \mathbb{R}^n$ such that $||q_1||_2 = 1$, tol
1. normalize $q_1 : q_1 = q_1/||q_1||_2$
2. initialize $\alpha_1 = q_k A q_k^T$
3. initialize $r_k = Aq_k - \alpha_k q_k$
4. initialize $\beta_1 = ||r_k||_2$
5. $k = 1$
9. while $k \leq n$ and $\beta_k >$ tol
10. $q_{k+1} = r_k/\beta_k$
11. $\alpha_{k+1} = q_{k+1}^T A q_{k+1}$
12. $r_{k+1} = Aq_{k+1} - \alpha_{k+1}q_{k+1} - \beta_k q_k$
13. $\beta_{k+1} = ||r_{k+1}||_2$
14. $k = k + 1$
15. end while
16. T=diag(α) + diag(β,1) + diag(β,-1)
Output: $T, Q = [q_1, ..., 1_k]$. Then one can compute the eigenvalues of and eigenvectors of the matrix T and retreive eigenvectors of A through Q.

10 Test results for Lanczos method

This section presents test results for the Lanczos method. Tests were carried out on the following random matrices:

- $C_1 \in \mathbb{R}^{(9,9)}$ symmetric positive definite
- $C_2 \in \mathbb{R}^{(9,9)}$ symmetric positive definite and sparse (sparsity 0.7)

After obtaining the tridiagonal matrix T similar to the original matrix C_1 with the Lanczos algorithm, we computed the eigenpairs of T and retrieved the eigenvectors of A.

10.1 Test matrix C_1

28.3276	4.2099	0	0	0	0	0	0	0
4.2099	10.2625	0.4083	0	0	0	0	0	0
0	0.4083	9.8692	0.3001	0	0	0	0	0
0	0	0.3001	10.2473	0.5167	0	0	0	0
0	0	0	0.5167	9.4844	0.1257	0	0	0
0	0	0	0	0.1257	9.4778	0.2414	0	0
0	0	0	0	0	0.2414	9.4750	0.1517	0
0	0	0	0	0	0	0.1517	9.2360	0.2272
0	0	0	0	0	0	0	0.2272	9.3758

Figure 8: Tridiagonal matrix obtained after applying Lanczos algorithm to C_1

	Lanczos
execution time	1.577240e-02
correctness of eigenvalues*	yes
max residual $Av_i - \lambda_i v_i$	1.270788e-03

Table 9: Test results for the computation of the eigenvalues of C_1

*with a tolerance of e-07

10.2 Sparse matrix C_2

	Lanczos
execution time	1.573350e-02
correctness of eigenvalues*	yes
max residual $Av_i - \lambda_i v_i$	2.080748e-04

Table 10: Test results for the computation of the eigenvalues of C_2

*with a tolerance of e-06

In general, the Lanczos method provides satisfactory results for the eigenpairs computation: eigenvalues are correct up to a tolerance of the order 10^{-7} (10^{-6} for the sparse case) and the residual for eigenvectors correctness is of the order 10^{-3} (10^{-4} in the sparse case), constituting the best approximation of eigenvectors for the methods tested so far. However, the major limitation of the method is its applicability only to symmetric positive definite matrices.
The execution time is low and it is interesting to notice that it does not diminish in the case of the sparse matrix.
In conclusion, results are considerable for the dense matrix C_1 and did not get worse when using the sparse matrix C_2, suggesting that the Lanczos method is suitable for this type of matrices.

11 Singular value decomposition

Singular values and singular vectors represent a generalization of eigenvalues and eigenvectors for rectangular matrices.

Theorem 6. *Let $A \in \mathbb{C}^{(m,n)}$ be a rectangular matrix. Then, there exists a unique diagonal matrix $\Sigma \in \mathbb{R}^{(m,n)}$ such that there exist $U \in \mathbb{C}^{(m,m)}$ and $V \in \mathbb{C}^{(n,n)}$ satisfying:*

- $A = U\Sigma V^H$
- $U^H U = I_m$
- $V^H V = I_n$
- *the diagonal elements of $\Sigma = diag(\sigma_1, ..., \sigma_p)$, with $p = min\{m, n\}$, satisfy:*

$$\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_p \geq 0, \ \sigma_i \in \mathbb{R}$$

Definition 6. *The σ_i are called singular values of A and $A = U\Sigma V^H \iff AV = U\Sigma$ is the singular value decomposition of A. Denoting u_i $i = 1, ..., m$ the columns of U and v_i $i = 1, ..., n$ the columns of V, it holds that $Av_i = \sigma_i u_i$ $\forall i = 1, ..., p$. Moreover, u_i are called left singular vectors and v_i right singular vectors of A.*

Theorem 7. *Let $A \in \mathbb{C}^{(m,n)}$ and $A = U\Sigma V^H$ its SVD decomposition. Assume that $k \in \mathbb{N}$ is such that $\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_k > \sigma_{k+1} = \sigma_{k+2} = ... = \sigma_p = 0$. Then*

- $A = \sum_{i=1}^{k} \sigma_i u_i v_i^H$ *and* $rank(A) = k$
- $Ker(A) = Span\{v_{k+1}, ..., v_n\}$
- $Im(A) = Span\{u_1, ..., u_k\}$
- $A^T A v_i = \sigma_i^2 v_i$ *and* $AA^T u_i = \sigma_i^2 u_i$ $\forall i = 1, ..., p$
- *If $m = n$ and A is symmetric, then $\sigma_i = |\lambda_i|$ $\forall i = 1, ..., n$ eigenvalues of A and $U = V = X$ matrix of eigenvectors of A*

Let us assume $A \in \mathbb{R}^{(m,n)}$ with $m \geq n$. To steps to follow in order to compute the singular value decomposition of A are:

1. compute Q and D such that $A^T A = QDQ^T$ where $D = diag(\sigma_1^2, ..., \sigma_p^2)$ is the matrix of eigenvalues and $Q = [v_1, ..., v_n]$ is the matrix of eigenvectors of $A^T A$.
2. Let $C = AQ \in \mathbb{R}^{(m,n)}$. Compute a permuted QR decomposition of C : $CP = UR$ in such a way that the diagonal elements of R are sorted in decreasing order.
3. define R_1 such that $R = (R_1, 0)^T$. Then $A = CQ^T = (URP^T)Q^T$ and $A^T A = Q(PR^T RP^T Q)^T = QDQ^T$ and $R_1^T R_1 = P^T DP$, so R_1 is a diagonal matrix and

   $$A = UR(QP)^T$$

   is the SVD decomposition of A with $\Sigma = R$ and $QP = V$

For 1. the calculation of eigenvalues and eigenvectors of A^TA is computed by turning A^TA into bidiagonal form, creating the orthogonal matrices P and H such that $PAH = B$ is bidiagonal. This is because A^TA is similar to B^TB through H, so the eigenvalues are the same and the eigenvectors can be retrieved.
In order to transform a matrix into its bidiagonal form, Householder matrices are used. The k-th step, when k-1 rows and columns are already in bidiagonal form, is the following:

- extract a_{k-1} and b_{k-1} from

$$A^{(k-1)} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & ... & 0 \\ 0 & ... & ... & & \\ & & \alpha_{k-1} & \beta_{k-1} & \\ \vdots & & & | & -\mathbf{b}_{k-1}^T- \\ & & & \mathbf{a}_{k-1} & \\ 0 & & & | & \mathbf{B}_{k-1} \end{bmatrix}$$

  where $\mathbf{a}_{k-1} \in \mathbb{R}^{m-(k-1)}$, $\mathbf{b}_{k-1} \in \mathbb{R}^{n-k}$ and $\mathbf{B} \in \mathbb{R}^{(m-k-2,n-k-2)}$

- compute the Householder vectors $\tilde{a}_{k-1} = a_{k-1} + sign(a_{k-1,1})||a_{k-1}||_2 e_1^{(m-(k-1))}$ and $\tilde{b}_{k-1} = b_{k-1} + sign(b_{k-1,1})||b_{k-1}||_2 e_1^{(n-k)}$

- build the Householder matrix

$$(*)\ P^{(k-1)} = \begin{bmatrix} \mathbf{I}_{k-1} & \\ & \mathbf{I}_{m-(k-1)} - 2\frac{(\tilde{a}_{k-1}\tilde{a}_{k-1}^T)}{||\tilde{a}_{k-1}||_2^2} \end{bmatrix}$$

- build the Householder matrix

$$(**)\ H^{(k-1)} = \begin{bmatrix} \mathbf{I}_k & \\ & \mathbf{I}_{n-k} - 2\frac{(\tilde{b}_{k-1}^T\tilde{b}_{k-1})}{||\tilde{b}_{k-1}||_2^2} \end{bmatrix}$$

- update the matrix

$$A^{(k)} = P^{(k-1)}A^{(k-1)}H^{(k-1)} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & ... & 0 \\ 0 & ... & ... & & \\ \vdots & & \alpha_k & \beta_k & \\ 0 & & & & \mathbf{B}_k \end{bmatrix}$$

11.1 Bidiagonalization algorithm

Input: A
$B = A$,$P = I_m$, $H = I_n$
for k=1,...n
$a = B(k : m, k)$

$a = a + sign(a(1))||a||_2 e_1^{(m-(k-1))}$
$\tilde{P} = (*)$
$B = \tilde{P}B$
$P = \tilde{P}P$
if $k \leq n-2$
$b = B(k, k+1:n)$
$b = b + sign(b(1))||b||_2 e_1^{(n-k)}$
$\tilde{H} = (**)$
$B = B\tilde{H}$
$H = H\tilde{H}$
end if
end for
Output: $B = PAH$ bidiagonal

11.2 SVD algorithm

Input: A
$[B, H] = bidiagonal(A)$
compute $[\tilde{Q}, D]$ eigenvectors and eigenvalues of $B^T B$
retrieve eigenvectors of $A^T A$ $Q = H\tilde{Q}$
$C = AQ$
compute the permuted QR decomposition of C: $CP = UR$
$V = QP$
$\Sigma = R$
Output: U, Σ, V

12 Test results for SVD method

This section presents test results for the SVD method. Tests were carried out on the following rectangular matrix:

- $D_1 \in \mathbb{R}^{(9,6)}$

After obtaining the bidiagonal matrix B, bidiagonal form of the original matrix D_1 with the bidiagonalization algorithm, we computed the SVD of D_1. To do so, the SVD algorithms computes the eigenpairs of the matrix $B^T B$, which is similar to $D^T D$. The eigenpairs of $B^T B$ were obtained using alternatively the QR method, the power method and the Matlab funcion eig.

```
-1.5732    3.3164   -0.0000   -0.0000   -0.0000   -0.0000
-0.0000    1.5370   -1.0664   -0.0000    0.0000   -0.0000
 0.0000    0.0000    0.4526   -0.7485   -0.0000    0.0000
-0.0000    0.0000    0.0000   -0.8261    0.1878   -0.0000
-0.0000    0.0000    0.0000    0.0000    0.5954   -0.3855
-0.0000    0.0000    0.0000    0.0000   -0.0000    0.2481
-0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
 0.0000   -0.0000    0.0000    0.0000   -0.0000    0.0000
 0.0000   -0.0000   -0.0000    0.0000   -0.0000   -0.0000
```

Figure 9: Bidiagonal matrix similar to to D_1

12.1 Test matrix D_1

	SVD with QR	**SVD with P.M.**	**SVD with eig**
execution time	5.770320e-02	4.138800e-03	2.274400e-03
max residual $Av_i - u_i\sigma_i$	7.904466e+00	2.090421e+00	1.446303e+00
correctness of singular values*	yes	yes	yes
correctness of eigenvalues of A^TA*	yes	yes	yes
max residual of $A^TAx_i - \lambda_i x_i$	less than 1e-04	less than 1e-04	less than 1e-04

Table 11: Test results for the computation of the singular values of D_1

*with a tolerance of e-07

In general, the SVD method yields results that align with the theoretical expectations concerning the obtained reduced matrix, which is bidiagonal as it should. Results obtained with the SVD method are similar, in terms of execution time and accuracy of the singular values and singular vectors, regardless of the usage of QR, power method or the Matlab function eig. Singular values results are particularly satisfying, being them correct up to a tolerance of the order 10^{-7}.

13 Conclusions

The aim of the report was to implement, test and compare some methods for the computation of eigenvalues and eigenvectors of matrices. Specifically, we implemented and tested several methods of different kinds:

- some for the computation of one specific eigenpair: Power method for the first one and Inverse Power method for the last one (with shifting and deflation for the successive ones).

- some for the computation of all eigenpairs through the computation of a similar and "easier" matrix: an upper hessenberg/triangular for the QR method (via the QR decomposition of the matrix) and a tridiagonal one for the Lanczos method.

Tests were carried out to asses the performance of such methods on different test matrices. The results reveal a very satisfying performance of all methods in terms of eigenvalues computation. Eigenvectors, on the other hand, are computed with a smaller accuracy, which could be improved.
Adjudicating which method is the most efficient is not a trivial task, since they can be exploited for different kinds of matrices. When working with symmetric and positive definite matrices, the Lanczos method results to be the most accurate, but its restriction of applicability to this specific type of matrices is a drawback. On the other hand, the QR method can be applied to a wide range of matrices, but its accuracy, especially for the computation of eigenvectors, is not as satisfying.
The Power and Inverse Power methods result to be highly accurate when used to compute a singular eigenpair, but while the eigenvalues accuracy is not affected by the iteration of the methods of all the eigenpairs, the computed eigenvectors become less precise, especially when using deflation. This loss in correctness is for shifting applied to power method, whose usage is restricted to symmetric matrices.
Finally, the implementation of a method for the singular value decomposition of a rectangular matrix was carried out and tested, resulting in a good approximation.